

MFA Client Authentication

User Guide for New Users
Secure Banking — ESP32-C6 + Windows PC

Secure Banking Project

June 19, 2026

Abstract

This folder implements a **two-factor net-banking login demo**: something you know (password on the PC) plus something you have (a PUF-backed hardware token on ESP32-C6). The raw PUF secret never leaves the chip; the PC only receives a cryptographic proof over USB serial. This guide walks a new user through setup, daily use, and troubleshooting.

Contents

1	What This Project Does	3
1.1	The three layers you must complete	3
2	What You Need	3
2.1	Hardware	3
2.2	Software	3
2.3	Install PC dependencies	4
3	Folder Layout	4
4	End-to-End Architecture	4
5	Step-by-Step Setup	5
5.1	Step 1: PUF enrollment on the ESP32 (Layer 1)	5
5.2	Step 2: Deploy the MFA server (replace <code>main.py</code>)	5
5.3	Step 3: List serial ports (optional)	5
5.4	Step 4: MFA device enrollment (Layer 2)	6
5.5	Step 5: Login with password + device MFA (Layer 3)	6
6	What Happens During Login	6
6.1	Two factors	6
6.2	Login sequence diagram	7
6.3	Inside the ESP32 during <code>MFA:AUTH</code>	7
7	UART Protocol Reference	7
7.1	Status values	7
7.2	Common error reasons	8
8	Files Created During Use	8
8.1	On ESP32 flash	8
8.2	On the PC	8

9	Command-Line Options	8
9.1	enroll_client.py	8
9.2	login_client.py	9
10	Troubleshooting	9
10.1	Quick health check	9
11	Security Notes (Lab vs. Production)	9
12	Relationship to Other Project Folders	10
13	Quick Start Checklist	10

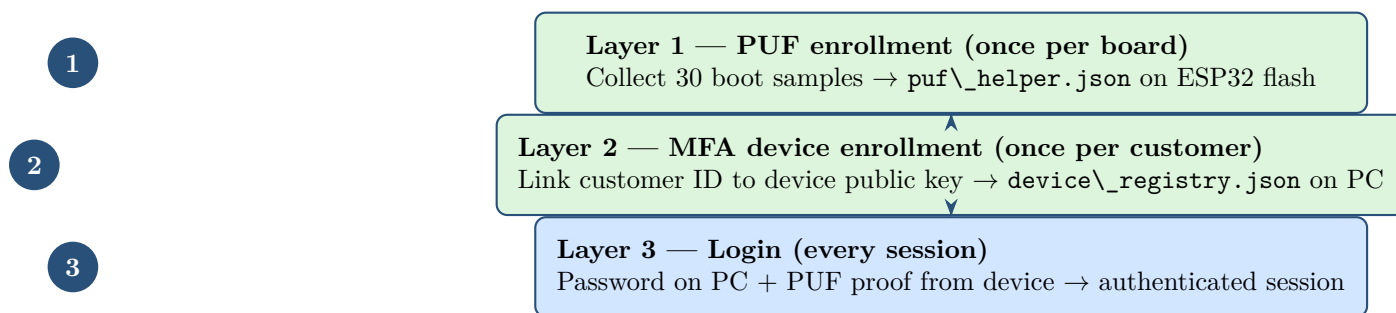
1 What This Project Does

The `MFA\_Client\_Authentication/` folder is a complete lab stack for **PUF-rooted multi-factor authentication (MFA)** over UART:

- **ESP32 side** (`esp32/`): reads the chip PUF at boot, derives a device X25519 key, and answers MFA commands on USB serial.
- **PC side** (`pc/`): simulates a bank login page, checks a password, then challenges the hardware device and verifies its response.

This is **not** production banking software. It demonstrates the architecture: password + hardware MFA, with keys rooted in silicon.

1.1 The three layers you must complete



Layer 1 uses the PUF enrollment script in `Code/main.py` (parent project). Layers 2 and 3 use this folder's `pc/` clients and `esp32/main.py`.

2 What You Need

2.1 Hardware

- ESP32-C6 development board with USB serial (example: COM5 on Windows)
- USB cable; note the correct COM port in Device Manager
- Windows PC (this guide uses PowerShell-style paths; Linux/macOS work with `/dev/tty*` ports)

2.2 Software

Table 1: Required software

Component	Where	Purpose
MicroPython (ESP32-C6)	On the board	Runs <code>boot.py</code> + <code>main.py</code>
Thonny or <code>mpremote</code>	PC	Flash <code>.py</code> files to ESP32
Python 3.8+	PC	Run enrollment and login clients
<code>pyserial</code>	PC	USB serial communication
L ^A T _E X (optional)	PC	Build this PDF from <code>userguide.tex</code>

2.3 Install PC dependencies

From the `pc/` directory:

```
cd MFA_Client_Authentication\pc
pip install -r requirements.txt
```

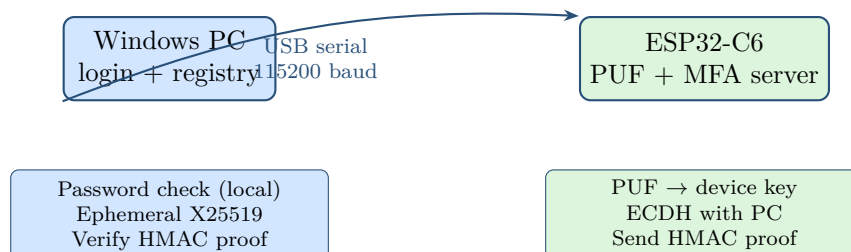
Only `pyserial` is required for the lab clients.

3 Folder Layout

```
MFA_Client_Authentication/
docs/
  userguide.tex      <- this document
esp32/
  boot.py            <- copy ALL .py files to the board
  main.py            <- reads LP-SRAM PUF once per boot
  mfa_auth.py        <- MFA UART server (run after PUF enrolled)
  key_derive.py      <- PUF reconstruct + HKDF
  device_identity.py <- PUF-rooted X25519 key
  mfa_store.py       <- login HMAC proof
  mfa_protocol.py    <- mfa_enrollment.json on flash
  puf_io.py, sketch.py, bch.py, hkdf.py, x25519.py, ...
  config.py          <- UART command constants
pc/
  enroll_client.py   <- UART command constants
  login_client.py    <- baud rate, file paths, HKDF labels
  uart_client.py     <- register device for a customer
  protocol.py        <- password + device MFA login
  mfa_auth.py        <- serial helpers
  x25519_pure.py     <- parse MFA responses
  device_registry.json <- verify HMAC proof
  requirements.txt   <- must match ESP X25519 math
                   <- bank-side device records
```

Rule of thumb: everything under `esp32/*.py` goes on the chip; everything under `pc/` stays on the computer.

4 End-to-End Architecture



Never sent over the wire: raw PUF data, stable PUF word, device private scalar, or the final PUF-derived AES key.

Sent over the wire: protocol text lines (status, enroll, auth challenge, HMAC proof). The PC learns only the device *public* key at enrollment time.

5 Step-by-Step Setup

5.1 Step 1: PUF enrollment on the ESP32 (Layer 1)

Before MFA works, the board must complete **PUF fuzzy extraction enrollment**. This is a **one-time** process per physical chip.

1. Copy PUF stack files to the board: at minimum `boot.py`, `puf\_state.py`, `puf\_io.py`, `bch.py`, `sketch.py`, `hkdf.py`, `config.py`, and `Code/main.py` from the parent project (rename or use as `main.py` on the board).
2. Reset the board. Each boot appends one LP-SRAM sample.
3. **Power-cycle 30 times** (`ENROLL_SAMPLES = 30`). Watch serial output: `sample 1/30`, `2/30`, ...
4. After sample 30, enrollment finalises and writes:
 - `/puf\_helper.json` — stable bit positions, BCH syndrome, salt
 - `./puf\_enrolled` — marker file
5. Confirm serial shows `=== ENROLLMENT COMPLETE ===`.

Important: Do not skip Layer 1. If `MFA:STATUS?` returns `puf_not_enrolled`, run PUF enrollment first. For a deep explanation of how runtime PUF reconstruction works, see `../../docs/puf\_enrollment\_vs\_runtime.tex` in the parent repo.

5.2 Step 2: Deploy the MFA server (replace `main.py`)

After PUF enrollment succeeds:

1. **Keep** on the board: `boot.py`, `puf\_helper.json`, `.puf\_enrolled`, and all crypto modules from `esp32/`.
2. **Replace** `main.py` with `MFA\_Client\_Authentication/esp32/main.py`.
3. **Remove** conflicting files from other demos if present (e.g. `pairing.json`, `secure\_protocol.py` from the secure UART demo).
4. Reset the board. Serial should show:

```
MFA PUF device server baud=115200
Status: puf_ready
```

5. Close Thonny or any serial monitor before running PC clients (only one program may open the COM port).

5.3 Step 3: List serial ports (optional)

```
python enroll_client.py --list-ports
```

Use the port name shown (e.g. `COM5`) in the next steps.

5.4 Step 4: MFA device enrollment (Layer 2)

Link a bank customer ID to this physical device:

```
cd MFA_Client_Authentication\pc
python enroll_client.py --port COM5 --customer CUST10042
```

Expected output:

```
Connected to COM5
Device status: puf_ready
Enrolling customer CUST10042 ...
Enrollment OK
  Customer : CUST10042
  Device pubkey: 6e75b5b8e10c3c2f6746959ff7337ee ...
  Saved to : device_registry.json
```

The PC stores the mapping in `device\_registry.json`. The ESP32 stores `/mfa\_enrollment.json` with the same customer ID and public key hex.

After this, `MFA:STATUS?` should return `mfa_enrolled`.

5.5 Step 5: Login with password + device MFA (Layer 3)

```
python login_client.py --port COM5 --customer CUST10042 --password demo1234
```

Add `-verbose` to see every serial line. Authentication on the device (PUF + X25519) may take **several seconds** on ESP32-C6; the client waits up to 120 s.

Successful login ends with:

```
=== LOGIN SUCCESS ===
Factors: password + PUF hardware device
Customer: CUST10042
Session : LOGIN-xxxxxxxx
```

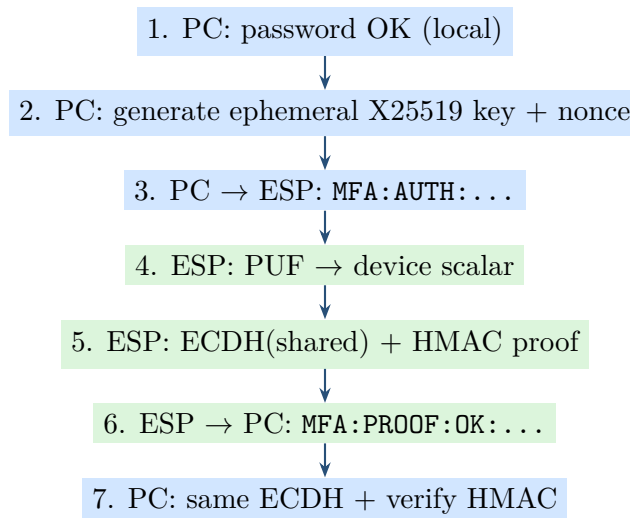
6 What Happens During Login

6.1 Two factors

Table 2: Authentication factors in this lab

Factor	Where	In this demo
Something you know	PC	Password (<code>-password</code> , min. 4 chars)
Something you have	ESP32	PUF-derived device key signs login proof

6.2 Login sequence diagram



6.3 Inside the ESP32 during MFA:AUTH

When `main.py` receives an auth command:

1. `boot.py` already captured `puf_state.raw` this boot.
2. `key\_derive.py` reconstructs the stable PUF word using `puf\_helper.json` and BCH correction.
3. `device\_identity.py` runs HKDF → X25519 device scalar (never stored in flash).
4. Device scalar + PC ephemeral public key → shared secret (ECDH).
5. `mfa\_auth.py` computes HMAC-SHA256(shared, transcript) where transcript includes login ID, customer ID, and nonce.
6. Response: MFA:PROOF:OK:<64-char-hex>.

The PC repeats ECDH with its ephemeral private key and the registered device public key, then verifies the same HMAC.

7 UART Protocol Reference

All lines are ASCII, terminated by `\n`. Baud rate: **115200** (`UART_BAUD` in `config.py`).

7.1 Status values

- `puf_not_enrolled` — run Layer 1 (PUF enrollment) first.
- `puf_ready` — PUF OK; run `enroll\_client.py` (Layer 2).
- `mfa_enrolled` — ready for `login\_client.py`.

Table 3: Commands and responses

PC sends	Device responds
MFA:STATUS?\n	MFA:STATUS:OK:puf_not_enrolled MFA:STATUS:OK:puf_ready MFA:STATUS:OK:mfa_enrolled
MFA:PUBKEY?\n	MFA:PUBKEY:OK:<64-hex> or MFA:ERR:...
MFA:ENROLL:<customer>\n	MFA:ENROLL:OK:<customer>:<64-hex-pubkey>
MFA:AUTH:<login_id>:<customer_id>:<proof>\n	MFA:WORKPHUB:<nonce>\n then MFA:PROOF:OK:<64-hex-hmac>
(any error)	MFA:ERR:<reason>

7.2 Common error reasons

MFA:ERR:...	Typical cause
bad_auth_format	Malformed auth line (wrong number of : fields)
not_mfa_enrolled	Layer 2 not done on device
customer_mismatch	-customer does not match device record
puf_not_enrolled	Missing puf_helper.json
reconstruct_failed	PUF noise too high (>10 bit errors)
auth_failed:...	Exception during X25519/HMAC on device
unknown_command	Garbage on serial or wrong main.py running

8 Files Created During Use

8.1 On ESP32 flash

File	When created
puf_samples.json	During Layer 1 only (deleted when done)
puf_helper.json	End of Layer 1
.puf_enrolled	End of Layer 1
mfa_enrollment.json	Layer 2 (enroll_client.py)

8.2 On the PC

File	Purpose
device_registry.json	Maps customer_id → device public key

9 Command-Line Options

9.1 enroll_client.py

```
python enroll_client.py --port COM5 --customer CUST10042
python enroll_client.py --list-ports
python enroll_client.py --port COM5 --baud 115200 --customer CUST10042
```

9.2 login_client.py

```
python login_client.py --port COM5 --customer CUST10042 --password demo1234
python login_client.py --port COM5 --verbose
python login_client.py --list-ports
```

10 Troubleshooting

Table 4: Common problems and fixes

Symptom	Fix
PermissionError on COM port	Close Thonny, serial monitors, and other Python clients. Only one opener.
ESP32 at REPL	Board not running MFA main.py. Deploy files and reset.
Timeout waiting for response	Use <code>-verbose</code> ; ensure MFA server printed startup banner.
puf_not_enrolled device PUF is ready but MFA not enrolled	Complete Layer 1 with <code>Code/main.py</code> , 30 power cycles. Run <code>enroll_client.py</code> before login.
MFA device proof verification FAILED	Often stale <code>device_registry.json</code> ; login client auto-updates live pubkey via <code>MFA:PUBKEY?</code> . Re-run enroll if needed.
unknown_command / garbage lines	Normal on COM open; client ignores non-MFA: lines. Persistent errors: wrong <code>main.py</code> on board.
Auth takes 10–30+ seconds	Expected on ESP32-C6 (pure-Python X25519). Wait; timeout is 120 s.
Mixed files from other demos	Remove <code>pairing.json</code> , secure UART <code>main.py</code> , etc.

10.1 Quick health check

With the board connected and MFA main.py running, open a serial terminal or use a one-liner:

```
python -c "import serial,time;s=serial.Serial('COM5',115200,timeout=1);time.sleep(1);s.write(b'MFA:STATUS?\n');print(s.readline())"
```

Replace COM5 with your port. Expect `MFA:STATUS:OK:mfa_enrolled` after full setup.

11 Security Notes (Lab vs. Production)

This demo is for learning and integration testing:

- Password check is **simulated** locally (any password ≥ 4 characters passes).
- There is no TLS, no bank server, and no account database.
- The device public key is stored in plain JSON on the PC.
- UART is trusted in the lab; production would use secure channels and attestation.

Production hardening would add: real credential verification, secure enrollment ceremony, rate limiting, secure element or attestation, and never storing long-term secrets in reversible form on the host.

12 Relationship to Other Project Folders

Folder	Role
Code/	PUF enrollment <code>main.py</code> (Layer 1)
uart_key_exchange/	Plaintext PUF key export over UART (earlier demo)
UART_Secure_Communication/	Pairing + encrypted key export (earlier demo)
MFA_Client_Authentication/	This guide — password + device MFA
docs/puf_enrollment_vs_runtime.tex	Deep dive: PUF enrollment vs runtime

Use **one demo's `main.py` at a time** on the board. Always keep `boot.py` and PUF helper files consistent with the same chip enrollment.

13 Quick Start Checklist

Flash MicroPython to ESP32-C6

Run PUF enrollment: 30 power cycles with `Code/main.py`

Confirm `puf\_helper.json` and `.puf\_enrolled` on board

Deploy all `MFA\_Client\_Authentication/esp32/*.py`; MFA `main.py` last

`pip install -r requirements.txt` in pc/

`python enroll_client.py -port COMx -customer CUST10042`

`python login_client.py -port COMx -customer CUST10042 -verbose`

See `=== LOGIN SUCCESS ===`

Build this PDF:

```
cd MFA_Client_Authentication/docs
pdflatex userguide.tex
pdflatex userguide.tex
```